

LAB-Project description

Introduction

The [pickle](#) module implements binary protocols for serializing and de-serializing a Python object structure. “Pickling” is the process whereby a Python object hierarchy is converted into a byte stream, and “unpickling” is the inverse operation, whereby a byte stream (from a [binary file](#) or [bytes-like object](#)) is converted back into an object hierarchy. Pickling (and unpickling) is alternatively known as “serialization”, “marshalling,” [\[1\]](#) or “flattening”; however, to avoid confusion, the terms used here are “pickling” and “unpickling”. [\[source\]](#)

I would like to understand how stable and correct pickle is: Does the same input always create the same (serialized) output?

We define the same input and output as hash-identical (equivalent is insufficient). This means an input must create the same pickle file under all circumstances.

Possible options are different [operating systems](#), different [Python \(but not pickle\) versions](#), floating point accuracy, and recursive data structures (e.g., json).

Please keep your mind open to other options.

Please make sure to consider [only the exact same input](#): if you alter the input (e.g., [x = 2+3](#) to [x = 3+2](#) or renaming variables), the results may be equivalent but are no longer identical when you hash the resulting pickle file (e.g., sha256).

Your task

Your task is to develop a test suite for the stability and correctness of the pickle.

You should use the [testing techniques](#) you learned in the lectures (equivalence partitioning, boundary values, fuzzing, etc).

Since the pickle library source code is available, you can also use white-box testing approaches as explained in the lectures (e.g., all-def, all-uses).

All results and findings must be reproducible.

Final report

You must submit a final report following what was discussed in the test plan and management lecture.

Your final report discusses:

- Your test suite, including applied strategies.
- The completeness of your test suite through a traceability matrix.
- The findings of your test suite.
- Why you decided to use (or not use) a testing technique (e.g., boundary value analysis)
- The limitations and shortcomings of your test suite.

Your final report does not exceed 8 pages and lists all team members and their contributions to the report and code in detail. Your code must comply with the PEP8 coding style guidelines and be available on GitHub, GitLab, or similar. Please provide a link to your repository.